

Optimizacija več-agentnega iskanja poti z uvedbo lokalnih vodij

Andraž Šošterič
andraz.sosteric@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

Nejc Fras
nejc.fras@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

David Lipavec
david.lipavec@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

Tadej Teršek
tadej.tersek@student.um.si
Faculty of Electrical
Engineering and Computer Science,
University of Maribor
Koroška cesta 46
SI-2000 Maribor, Slovenia

POVZETEK

V tem delu obravnavamo problem večagentnega iskanja poti MAPF, kjer primerjamo centralizirane in decentralizirane algoritme, ter predlagamo svoj pristop z lokalnimi vodijami (*local leaders*). Algoritem agente dinamično razporeja v lokalne skupine z vodji, ki določajo prioriteto pri razreševanju konfliktov. Vsak agent pa neodvisno načrtuje pot z algoritmom A^* .

MovingAI in POGEMA. Local Leaders dosledno dosega nižji SoC kot LaCAM in PBS ($\approx 9\%$ pri 64 agentih), hkrati pa je 100–400× hitrejši od MATS-LP pri lifelong scenarijih.

KLJUČNE BESEDE

centralizirano načrtovanje poti, distribuirano načrtovanje poti, usklajevanje agentov, izogibanje trkom, optimizacija poti

1 UVOD

V tem članku obravnavamo problem več-agentnega iskanja poti (Multi-Agent Path Finding / MAPF), ki se v praksi pojavlja v številnih domenah, kot so robotizirana skladišča, avtonomna vozila, inteligentni transportni sistemi in drugi logistični sistemi. Gre za temeljni problem koordinacije več avtonomnih enot v skupnem prostoru, kjer mora vsak agent doseči svoj cilj brez trkov z drugimi agenti ali ovirami.

Cilj problema je načrtovati poti za množico agentov v diskretnem prostoru tako, da se izognemo medsebojnim konfliktom in hkrati optimiziramo določene metrike, kot sta skupni čas izvajanja (ang. *makespan*) ali vsota dolžin poti (ang. *sum of costs*) [10].

Repozitorij z izvirno kodo projekta je na voljo na našem github repozitoriju ¹.

V nadaljevanju bomo primerjali centralizirane metode, katerih predstavnik sta LaCAM[6] in PBS[5], ki načrtujejo poti za vse agente z uporabo globalnega pogleda na problem, ter decentralizirane pristope, kot sta Follower[1] in MATS-LP[8], kjer posamezni agenti sprejemajo odločitve lokalno brez centralnega nadzora in imajo pregled le nad svojo okolico.

Na ta način bomo analizirali ključne razlike med obema paradigama z vidika učinkovitosti, skalabilnosti in kakovosti rešitev. Centralizirani pristopi praviloma zagotavljajo bolj optimalne rešitve, vendar slabše skalirajo z naraščajočim številom agentov, medtem ko decentralizirani pristopi omogočajo boljšo razširljivost, vendar pogosto na račun kakovosti rešitev.

Kot glavni referenčni algoritem izberemo delo *LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding* [6], ki predstavlja sodoben in učinkovit centraliziran pristop k reševanju problema MAPF. Izbrani članek služi kot referenčna osnova za našo analizo, saj ponuja dobro ravnotežje med učinkovitostjo in skalabilnostjo ter omogoča relativno enostavno reprodukcijo eksperimentov.

LaCAM bomo uporabili kot predstavnika centraliziranih metod, s katerim bomo primerjali decentralizirane pristope, kot sta Follower in MATS-LP, ter tako sistematično analizirali razlike med globalnim in lokalnim načrtovanjem poti.

1.1 Definicija problema

Stern et al. [10] definirajo problem več-agentnega iskanja poti (Multi-Agent Path Finding - MAPF) kot trojico

$$\langle G, s, t \rangle,$$

kjer je $G = (V, E)$ neusmerjen graf, pri čemer V predstavlja množico vozlišč, E pa množico povezav.

Podana je množica agentov $A = \{1, 2, \dots, k\}$. Funkcija

$$s : \{1, \dots, k\} \rightarrow V$$

preslika vsakega agenta v njegovo začetno vozlišče, funkcija

$$t : \{1, \dots, k\} \rightarrow V$$

pa v njegovo ciljno vozlišče.

Naj bo $\pi_i[t] \in V$ položaj agenta i v diskretnem času $t \in \mathbb{N}_{\geq 0}$. Pri vsakem koraku t se agent i lahko premakne v sosednje vozlišče ali ostane na trenutnem, tj.

$$\pi_i[t+1] \in \text{neigh}(\pi_i[t]) \cup \{\pi_i[t]\},$$

kjer je $\text{neigh}(v)$ množica vozlišč, sosednjih vozlišču $v \in V$ [6].

¹<https://github.com/Asos75/MultiAgent>

Po Okamuri [6] se mora agent izogibati naslednjim vrstam konfliktov:

- **Vozliščni konflikt:** Do vozliščnega konflikta (ang. vertex conflict) pride, če nameravata π_i in π_j zasedeti isto vozlišče v istem trenutku, torej obstaja časovni korak t , da velja $\pi_i[t] = \pi_j[t]$. [10]
- **Zamenjalni konflikt:** Do zamenjalnega konflikta (ang. swap conflict) pride, če nameravata π_i in π_j zamenjati svoji poziciji v istem trenutku, torej obstaja časovni korak t , da velja $\pi_i[t] = \pi_j[t + 1] \wedge \pi_i[t + 1] = \pi_j[t]$ [10].

Rešitev MAPF za instance, omenjene zgoraj, je množica brezkonfliktnih poti $\{\pi_1, \dots, \pi_k\}$ takih, da obstaja časovni korak $T \in \mathbb{N}_{\geq 0}$, kjer agent doseže cilj. [6]

2 SORODNA DELA

V tem razdelku izpostavimo ključna dela s področja več-agentnega iskanja poti, ki dopolnjujejo in razširjajo pogled iz uvoda. Pri vsakem delu na kratko povemo, kaj je njegova glavna ideja in kakšen doprinos ima k našemu eksperimentu.

2.1 Centralizirani pristopi

EECBS [4] je nadgradnja klasičnega CBS, ki ohrani dvonivojsko iskanje, hkrati pa z *Explicit Estimation Search* in uteževanjem omogoča nadzorovano suboptimalnost (uporabnik nastavi kompromis med kakovostjo in časom). Avtorji poročajo o bistveno boljši skalabilnosti na večjih instancah ob majhni, dokazljivo omejeni degradaciji kakovosti rešitve. EECBS uporabimo kot referenčno izhodišče za razumevanje kompromisa »hitrost vs. kvaliteta« pri centraliziranih metodah; njegove ideje utemeljijo, zakaj v evalvaciji poleg kakovosti (SoC/makespan) spremljamo tudi čas izvajanja in skalabilnost.

CBSH2 [3] izboljša CBS z močnejšimi heuristikami in tehniko *disjoint splitting*, ki z uvedbo pozitivnih omejitev zmanjša podvajanje iskanja in s tem velikost iskalnega drevesa. Rezultat je hitrejšo reševanje in boljša skalabilnost ob ohranjeni rešitveni kakovosti tipične za CBS-družino. CBSH2 obravnavamo kot dodatno referenco znotraj centraliziranih metod, ki pokaže, da so izboljšave pogosto posledica boljšega upravljanja konfliktov; to nam pomaga pri interpretaciji rezultatov LaCAM/PBS (kjer tudi ciljamo na učinkovitejšo obravnavo konfliktov), če opazimo razlike predvsem v času in ne v SoC.

2.2 Decentralizirani pristopi

Follower [1] je decentraliziran pristop za *lifelong* MAPF, ki kombinira (i) heuristični planer (npr. A^* nad statičnimi ovirami), ki generira »razpršene« poti z namenom zmanjšanja zasičenosti ozkih grl, in (ii) naučeno politiko sledenja, ki lokalno reagira na druge agente in se izogiba trkom. Izvajanje je povsem lokalno (brez centralnega koordinatorja), zato metoda dobro skalira, vendar je kakovost poti odvisna od lokalnih informacij in stabilnosti naučene politike. *Follower* je eden naših glavnih decentraliziranih primerjalnih algoritmov; z njim ovrednotimo, kako se učeni, lokalni pristopi primerjajo s centraliziranimi LaCAM/PBS in z našim hibridnim pristopom z lokalnimi vodji.

Decentralized Monte Carlo Tree Search for Partially Observable Multi-agent Pathfinding [9] predlaga decentralizirano reševanje *lifelong* MAPF pri delni opazljivosti: vsak agent iz lokalnih opazovanj zgradi intrinzični (lokalni) MDP in na njem izvaja MCTS, ki ga dodatno usmerja nevronska politika. S simulacijami v drevesu agenti ocenjujejo kratkoročne interakcije z bližnjimi agenti, koordinacija pa nastaja implicitno (brez eksplicitne komunikacije ali centralnega koordinatorja), kar vodi do dobre skalabilnosti. delo utemeljuje našo izbiro decentraliziranih primerjalnih metod na POGEMA (delna opazljivost) in je neposredno relevantno za komponento MATS-LP v naši primerjavi, kjer želimo videti, ali »lookahead« (MCTS) izboljša pretočnost in robustnost glede na bolj reaktivne politike.

PRIMAL2 [2] je zgodnje, vplivno delo, ki pokaže, da je mogoče MAPF reševati z decentraliziranimi politikami na osnovi lokalnih opazovanj, naučenimi s kombinacijo *imitation learning* (iz centraliziranih planerjev) in *reinforcement learning*. Pristop zelo dobro skali, vendar pogosto žrtvuje optimalnost in lahko povzroča neusklajenosti v gostih scenarijih. *PRIMAL2* uporabljamo kot kontekst za interpretacijo rezultatov učenih decentraliziranih metod (Follower/MATS-LP/SCRIMP): če pri njih opazimo slabši SoC, je to pričakovan trade-off, ki ga literatura že izpostavlja.

SCRIMP [11] uvede eksplicitno komunikacijo med agenti z uporabo transformer arhitekture in pozornostnega mehanizma, kar omogoča učinkovitejšo koordinacijo tudi pri zelo omejenem vidnem polju. S tem pristop preseže omejitve povsem nekomunikacijskih lokalnih politik, hkrati pa ohrani decentralizirano izvajanje in dobro skalabilnost. *SCRIMP* služi kot motivacija za naš »hibridni« pogled (lokalni vodje): kaže, da lahko dodaten koordinacijski signal (komunikacija ali posredno usklajevanje) izboljša pretočnost; naš pristop to doseže z lažjim mehanizmom vodij, ki ga lahko primerjamo z relacijo med čisto lokalnimi in bolj koordiniranimi pristopi.

3 METODOLOGIJA

V tem poglavju bomo predstavili metodologijo vrednotenja in primerjanja izbranih algoritmov za problem večagentnega iskanja poti. Cilj eksperimentalnega dela je primerjati centralizirane pristope, kot sta LaCAM in PBS, z decentraliziranimi pristopi Follower in MATS-LP ter ovrednotiti predlagani pristop z lokalnimi vodji.

3.1 Eksperimentalno okolje

Za evalvacijo uporabljamo knjižnico POGEMA [7], ki ponuja standardizirano platformo za primerjavo MAPF algoritmov v delno opazljivih okoljih – z generatorjem instanc, enotnimi metrikami in protokolom izvajanja. To kombiniramo z *MovingAI* [10] zbirko zemljevidov, ki zagotavlja reprezentativne testne scenarije. POGEMA je dostopna kot standardna Python knjižnica, implementacije posameznih algoritmov pa so pridobljene iz uradnih repozitorijev avtorjev. Ker različni algoritmi uporabljajo različne vhodne formate in interne predstavitve okolja, smo za vsako implementacijo razvili poseben prilagoditveni sloj (*adapter*), ki omogoča njeno vključitev v skupno ogrodje.

3.2 Metrike

Sum-of-Costs (SoC) je vsota časovnih korakov, ki jih vsi agenti porabijo za doseg svojih ciljnih vozlišč [10]:

$$\text{SoC} = \sum_{i=1}^k |\pi_i|,$$

kjer je $|\pi_i|$ dolžina poti agenta i . Nižja vrednost pomeni učinkovitejšo rešitev.

SoC je ena izmed najpogostejše uporabljenih metrik v MAPF literaturi [10], kar omogoča neposredno primerjavo z obstoječimi deli. Metrika meri skupni strošek kot vsoto dolžin poti vseh agentov in s tem neposredno zajame celostno učinkovitost sistema.

Metrika je še posebej primerna za one-shot scenarije, saj minimizacija SoC ustreza cilju minimizacije skupne porabe časa oziroma virov. Hkrati kaznuje neučinkovite poti posameznih agentov, kar spodbuja iskanje globalno učinkovitih rešitev. SoC bomo zato uporabili za vrednotenje algoritmov LaCAM in PBS na zbirki MovingAI.

Makespan je čas, ki ga porabi najdaljša pot med vsemi agenti, torej maksimum nad vsemi $|\pi_i|$ [10]:

$$\text{makespan} = \max_{i \in \{1, \dots, k\}} |\pi_i|.$$

Metrika odraža skupno trajanje reševanja instance in je primerna za scenarije, kjer je ključen vzporedni zaključek vseh agentov. Makespan je uveljavljena metrika v MAPF literaturi [10] in je še posebej relevantna v praktičnih aplikacijah, kjer je ključen vzporedni zaključek vseh agentov.

Makespan je pri primerjavi centraliziranih algoritmov nujna dopolnitev k SoC, saj sama vsota stroškov namreč ne razkrije, ali posamezen agent morda bistveno zavlačuje celotno skupino. Z vključitvijo obeh metrik tako dobimo celovitejšo in bolj zanesljivo oceno učinkovitosti algoritma v realnih pogojih.

Stopnja uspeha (Success Rate) meri delež instanc, pri katerih vsi agenti uspešno dosežejo svoje cilje znotraj predvidenega časovnega horizonta [7].

Stopnja uspeha je primarna metrika platforme POGEMA in skupaj s pretokom (ang. throughput) tvori osnovo za vrednotenje na tej platformi [7]. Zasnovana je posebej za vrednotenje decentraliziranih metod v delno opazljivih okoljih. Metrika je za naš eksperiment ključna, saj za razliko od SoC in makespana meri tudi robustnost algoritma oziroma njegovo sposobnost, da instanco v celoti reši kljub omejenim lokalnim opazovanjem.

Čas načrtovanja merimo kot skupni čas izvajanja algoritma na posamezni instanci. Ta metrika je še posebej pomembna pri primerjavi med kategorijama, saj centralizirani načrtovalci tipično porabi več časa za globalno načrtovanje, kot decentralizirani agenti.

Čas načrtovanja je v praktičnih aplikacijah enako pomemben kot kakovost rešitve, saj algoritem z optimalno potjo, a dolgim časom načrtovanja, v realnih sistemih ni uporaben. Ta metrika nam tako omogoča primerjavo med pristopoma z vidika razširljivosti in praktične uporabnosti.

3.3 Protokol primerjave

Primerjavo bomo izvedli na dveh ravneh:

- (1) **Znotraj kategorije:** LaCAM in PBS primerjamo na MovingAI zbirki po metrikah SoC, makespan in čas načrtovanja. Follower in MATS-LP primerjamo na POGEMA zbirki po metrikah throughput in stopnja uspeha.
- (2) **Med kategorijama na skupni zbirki:** Obe kategoriji bomo primerjali med seboj na skupni zbirki, kjer bomo primerjali prej navedene metrike.

3.4 Uvedba lokalnih vodij

Glavni prispevek tega dela je predlagana nadgradnja obstoječih pristopov, ki uvaja koncept *lokalnih vodij*. Gre za algoritem *Local Leaders*, ki združuje prednosti centraliziranega in decentraliziranega pristopa. Algoritem deluje lokalno, na ravni agenta, brez vnaprejšnjega načrtovanja in deluje v petih korakih (sl. 1).

1. *Oblikovanje skupin.* Na začetku vsakega koraka, se agenti razdelijo v skupine. Pravilo za deljenje je preprosto in deterministično: Agent i postane *vodja*, če nobeden od agentov z nižjim ID-jem ne leži znotraj razdalje r_{leader} (razdalja se izračuna po Manhhattanski metriki). Procesiranje agentov, od najnižjega do najvišjega ID-ja zagotovi konsistentno particijo brez dvomnosti. Pri tem se pa particija se spremeni vsak korak, tako da je članstvo v skupini dinamično glede na trenutno postavitev.

2. *Željena celica (A^*).* Vsak agent zase neodvisno izvede algoritem A^* od svojega položaja, do cilja, pri čimer gleda le na statične ovire. Agent na tej točki ignorira ostale agente. Rezultat tega koraka, je prva celica najkrajše poti, to je celica kamor se agent *želi* premakniti. Agenti, ki so obstali $\geq t_{\text{esc}}$ korakov preidejo v *način pobega*, kjer namesto A^* izberejo naključno prosto sosednjo celico, s čimer prekinajo tesnobne zastoje v hodnikih.

3. *Razrešitev konfliktov v dveh fazah.* Razrešitev poteka v dveh eksplicitnih fazah, ki ločita konflikte *znotraj* skupin od konfliktov *med* skupinami.

Faza A — znotrajskupinska razrešitev (vodja posreduje). Vsak vodja neodvisno razrešuje konflikte znotraj svoje skupine z lokalno karto zavez. Obdelava članov znotraj skupine poteka v naslednjem vrstnem redu:

- (1) agenti v načinu pobega (potrebujejo prednost, da se sprostijo iz zastoja),
- (2) preostali člani, urejeni po bližini do cilja (bolj nujni agenti prej), pri enakosti po ID-ju.

Za vsakega člana vodja preveri *vozliščni* (oba agenta v isto celico) in *zamenjalni* konflikt (agenta zamenjata poziciji). Agent, ki pride na vrsto pozneje, se preusmeri v najboljšo prosto alternativo. Rezultat tega koraka so začasni položaji *provisional[i]* vsakega agenta.

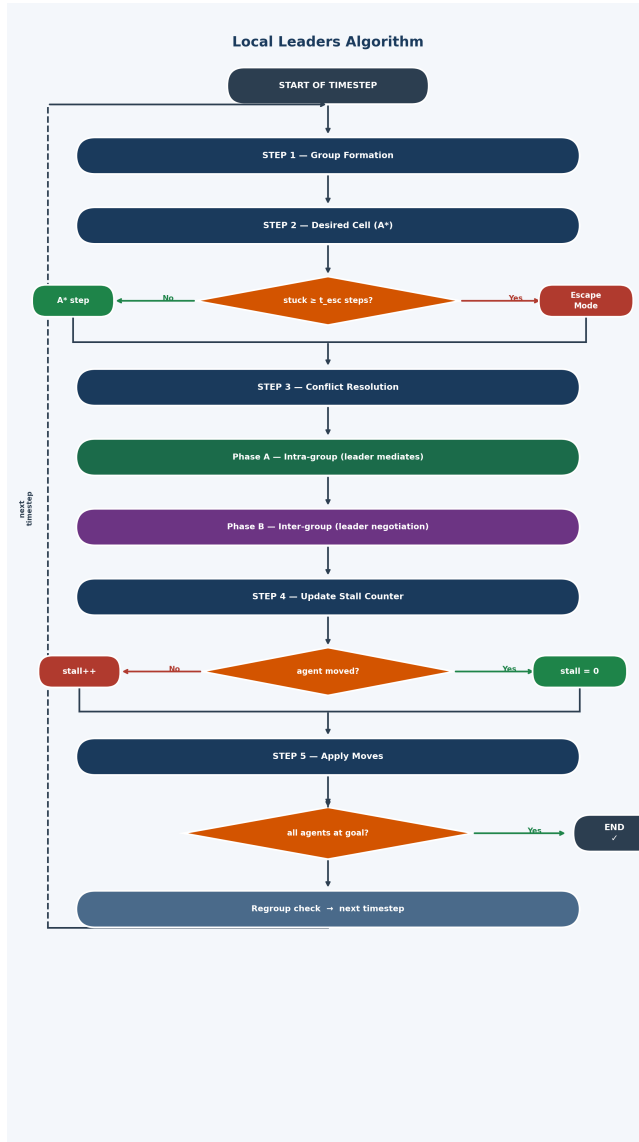
Faza B — medskupinska razrešitev (pogajanje vodij). Agenti v načinu pobega najprej dobijo globalni predprehod, ne glede na prioriteto svoje skupine. Nato se skupine zavežejo v skupno globalno karto v naslednjem vrstnem redu:

- (1) skupine z največjim številom običanih agentov,
- (2) pri enakosti: skupina, katere vodja je bližje svojemu cilju,
- (3) pri enakosti: skupina z nižjim ID-jem vodje.

Ko agent iz nižjeprioritene skupine zahteva celico, ki jo je višjeprioritena skupina že zavzela, se agent preusmeri — to je implicitno pogajanje med dvema vodjema.

4. *Posodobitev števca zastoja.* Agent, ki se ni premaknil, poveča števec zastoja; agent, ki se je premaknil, ga ponastavi na nič.

5. *Implementacija.* Algoritem je implementiran v Pythonu za integracijo z okoljem POGEMA in MovingAI, ter v C++ (prek pybind11) za hitrejšo izvajanje. Parametri uporabljeni za testiranje so $r_{\text{agent}} = 11$ (vidno polje agenta), $r_{\text{leader}} = 17$ (vidno polje vodje), $t_{\text{esc}} = 1$ (prag pobega) in $t_{\text{regroup}} = 5$ (interval ponovnega združevanja skupin).



Slika 1: Diagram poteka algoritma Local Leaders: dinamično oblikovanje skupin, določitev vodje, A* načrtovanje in dvofazna razrešitev konfliktov (znotraj skupinska posredovanost vodje, nato medskupinsko pogajanje vodij).

Lokalni vodja nima globalnega pogleda nad celotnim okoljem, temveč razpolaga le z informacijami o agentih znotraj svoje skupine

in njihove neposredne okolice. Na ta način se bistveno zmanjša kompleksnost načrtovanja v primerjavi s centraliziranimi algoritmi, hkrati pa se ohrani višja stopnja koordinacije kot pri popolnoma decentraliziranih pristopih.

Hipoteza našega dela je, da lahko takšen hibridni pristop doseže boljše razmerje med kakovostjo rešitev in časom načrtovanja kot obstoječi centralizirani ali decentralizirani pristopi posebej. Pričakujemo nižji *makespan* in višjo stopnjo uspeha kot pri decentraliziranih metodah ter bistveno krajši čas načrtovanja kot pri popolnoma centraliziranih algoritmi.

4 EKSPERIMENT IN ANALIZA REZULTATOV

Po začetni validaciji implementacij smo zasnovali lasten eksperiment, ki primerja centralizirane, decentralizirane in hibridni pristop z lokalnimi vodji. Namen experimenta je preveriti ali lahko uvedba lokalnih vodij izboljša razmerje med kakovostjo rešitev in časom načrtovanja.

Experimente izvajamo na standardiziranih zemljevidih iz zbirk MovingAI in POGEMA, na različnih velikostih zemljevidov, gostotah agentov in scenarijih. Tu so najbolj zanimivi scenariji z visoko gostoto agentov, kjer se razlike med rešitvami najbolj pokažejo.

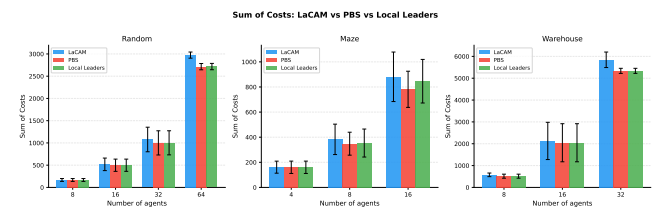
4.0.1 *Testne instance.* Uporabljamo tri glavne tipe okolij:

- **Odperte mreže** (*random grids*) – okolja z malo ovirami, kjer je večina konfliktov posledica visoke gostote agentov.
- **Labirinti** (*mazes*) – okolja z ozkimi prehodi in številnimi lokalnimi ozkimi grli, kjer je koordinacija agentov bistveno zahtevnejša.
- **Skladišni scenariji** (*warehouse maps*) – realistične postavitve, podobne robotiziranim skladiščem, kjer se agenti pogosto srečujejo na križiščih in v ozkih hodnikih.

Za vsako vrsto okolja izvajamo eksperimente pri različnih številih agentov (od 20 do 1000 agentov, odvisno od velikosti zemljevida), z različnimi konfiguracijami. Vsaka konfiguracija se izvede večkrat z različnimi scenariji, ki vključujejo začetne in ciljne pozicije agentov, rezultati pa se povprečijo, da zmanjšamo vpliv posameznih naključnih primerov.

4.1 Vsota stroškov (SoC)

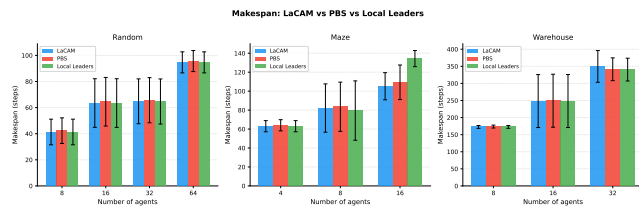
Slika 2 prikazuje vsoto stroškov za LaCAM, PBS in Local Leaders na treh tipih zemljevidov pri različnih številih agentov.



Slika 2: Vsota stroškov (SoC): LaCAM, PBS in Local Leaders na naključnih, labirintnih in skladišnih zemljevidih. Napake kažejo standardno deviacijo čez 3 semena.

Ključna ugotovitev je, da tako PBS kot Local Leaders dosledno dosegata nižji SoC kot LaCAM pri vseh tipih zemljevidov. Na naključnih zemljevidih pri 8 agentih so vrednosti skoraj identične (LaCAM: 168, PBS: 167, LL: 167), pri 64 agentih pa Local Leaders doseže SoC 2716, PBS pa 2715, medtem ko LaCAM doseže 2974, kar je $\approx 9\%$ manj za oba. Na skladiščnih zemljevidih je prihranek pri 32 agentih $\approx 9\%$ (PBS: 5337, LL: 5338 vs. LaCAM: 5844). Na labirintnih zemljevidih PBS doseže SoC 782 pri 16 agentih (vs. LaCAM 881, $\approx 11\%$ manj), Local Leaders pa 846 ($\approx 4\%$ manj, le pri uspešnih instancah). Rezultat je nekoliko presenetljiv, saj LaCAM minimizira SoC globalno, medtem ko PBS in Local Leaders delujeta lokalno oziroma s prioritarno razrešitvijo, vzrok tiči v tem, da algoritma Local Leaders in PBS iščeta najkrajše poti, medtem ko LaCAM išče zagotovljeno rešitev.

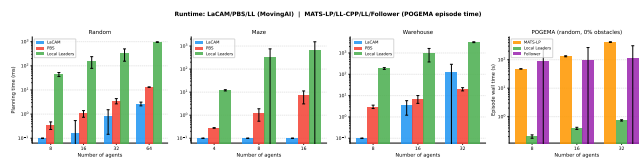
4.2 Makespan



Slika 3: Makespan: LaCAM, PBS in Local Leaders. Na naključnih in skladiščnih zemljevidih so vsi trije algoritmi praktično identični; na labirintih PBS doseže makespan 109 pri 16 agentih (vs. LaCAM 105), Local Leaders pa 134, kar je posledica delno neuspešnih instanc.

Slika 3 kaže, da je makespan vseh treh algoritmov na naključnih in skladiščnih zemljevidih skoraj enak. Na naključnih zemljevidih z 8 agenti so vrednosti identične (LaCAM: 41, PBS: 42, LL: 41), pri 64 agentih pa vsi dosežejo makespan 95–96. Na labirintih PBS ohranja 100 % uspešnost in doseže makespan 109 pri 16 agentih (LaCAM: 105), torej le minimalno višje. Izjema je Local Leaders: pri 16 agentih doseže makespan 134, ker uspešno reši le 50 % instanc (povprečje zajema le uspešne poskuse). PBS tako zapolni vrzel, ki jo Local Leaders pušča v scenarijih z ozkimi hodniki.

4.3 Čas izvajanja

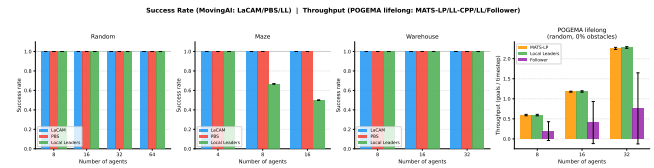


Slika 4: Čas izvajanja v logaritemski skali. Levi trije paneli: čas načrtovanja LaCAM, PBS in Local Leaders na MovingAI (ms). Desni panel: čas celotne epizode MATS-LP vs. Local Leaders vs. Follower na POGEMA (s, naključni zemljevidi, 0 % ovir).

Slika 4 razkrije eno ključnih razlik med pristopi. Na MovingAI LaCAM porabi manj kot 1 ms za manjše instance in do 128 ms za 32 agentov na skladiščnih zemljevidih. PBS je le malce počasnejši: 0,3–20 ms za vse scenarije, torej blizu LaCAM, a bistveno hitrejši od Local Leaders, ki deluje *online* (A^* za vsakega agenta pri vsakem koraku) in porabi 44–961 ms za naključne ter 188–3203 ms za skladiščne scenarije. PBS tako ponuja najboljše razmerje med kakovostjo rešitve in hitrostjo med vsemi tremi one-shot algoritmi.

Desni panel primerja čas epizode na POGEMA. Tu je razlika dramatična: MATS-LP potrebuje 48 s (8 agentov) do 416 s (32 agentov) za eno epizodo 512 korakov. Local Leaders opravi isto epizodo v 0,26–0,97 s, kar je **100–400×** hitreje. Follower (naučena RL politika) leži med njima s 2–7 s na epizodo.

4.4 Stopnja uspeha in pretočnost na POGEMA

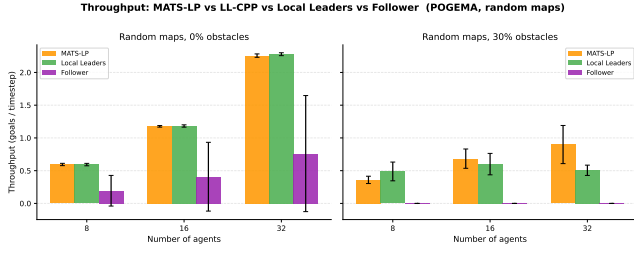


Slika 5: Levo: stopnja uspeha LaCAM, PBS in Local Leaders na MovingAI one-shot scenarijih. Desno: lifelong pretočnost MATS-LP, Local Leaders in Follower na POGEMA (naključni zemljevidi, 0 % ovir).

Slika 5 prikazuje dve dimenziji primerjave. V levem delu so stopnje uspeha na MovingAI: na naključnih in skladiščnih zemljevidih vsi trije algoritmi dosežejo 100 % pri vseh gostotah agentov. Na labirintnih zemljevidih LaCAM in PBS vedno uspesta, medtem ko Local Leaders dosega le 67 % (8 agentov) oziroma 50 % (16 agentov). PBS tukaj izrazito prednjači, ozkih hodnikov ne rešuje reaktivno, ampak z optimalnim iskanjem, ki zagotovi pot za vsakega agenta. To je primarna prednost PBS pred Local Leaders v gostih labirintnih scenarijih.

Desni panel prikazuje lifelong pretočnost (cilji/korak) na POGEMA. Rezultati prikazujejo C++ implementacijo Local Leaders, ki je po pretočnosti primerljiva s Python različico, a bistveno hitrejša pri izvedbi. MATS-LP doseže 0,61 (8 agentov), 1,18 (16) in 2,26 (32 agentov). Local Leaders je tik zadaj: 0,59, 1,18 in 2,28, kar je 97–101 % pretočnosti MATS-LP. Follower dosega nižje vrednosti (0,36, 0,76, 1,41).

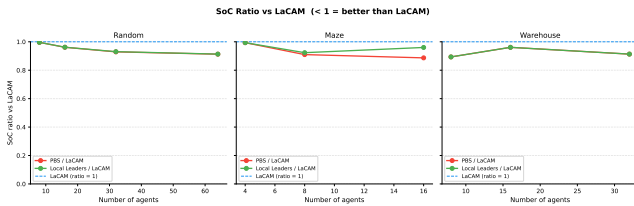
4.5 Primerjava pretočnosti pri 30 % ovirah



Slika 6: Lifelong pretočnost MATS-LP, Local Leaders in Follower na naključnih zemljevidih pri 0 % (levo) in 30 % (desno) gostoti ovir. Rezultati Local Leaders so iz C++ implementacije. Pri 30 % ovirah Local Leaders ostaja konkurenčen pri manjšem številu agentov, a zaostaja pri 32 agentih.

Slika 6 kaže, kako gostota ovir vpliva na vsak algoritem. Pri 0 % ovirah so vse tri krivulje blizu skupaj in dobro skalirajo z naraščajočim številom agentov. Pri 30 % ovirah se slika nekoliko spremeni: MATS-LP ohranja razumno pretočnost (0,36–0,90), Follower je pri 32 agentih primerljiv z MATS-LP (0,94). Local Leaders s posodobitvijo parametrov ($t_{esc} = 1$) in dvofazno razrešitvijo konfliktov bistveno izboljša pretočnost pri gostih ovirah: pri 8 agentih doseže 0,49 (vs. MATS-LP 0,36), pri 16 agentih 0,60, pri 32 agentih pa 0,51 (vs. MATS-LP 0,90). Agresivnejši mehanizem pobega in posredovanje vodje znotraj skupin učinkoviteje razrešujeta ciklične zastoje v ozkih hodnikih, čeprav je pri visokem številu agentov MATS-LP še vedno boljši.

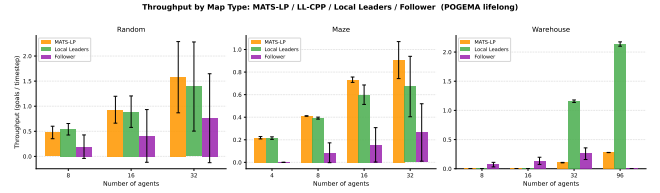
4.6 SoC overhead razmerje



Slika 7: Razmerje SoC (PBS / LaCAM in Local Leaders / LaCAM) po tipu zemljevida. Vrednost pod 1 pomeni nižji SoC kot LaCAM.

Slika 7 prikazuje razmerje SoC za oba algoritma glede na LaCAM. Oba, PBS in Local Leaders, pri vseh tipih zemljevidov in gostotah agentov dosežeta nižji SoC kot LaCAM (razmerje $< 1,0$). Na naključnih zemljevidih razmerji PBS in Local Leaders sledita podobni krivulji, ki pada z naraščajočim številom agentov: od $\approx 0,994$ (8 agentov) do $\approx 0,913$ (64 agentov), vrednosti so tako podobne, da se črti praktično prekrivata. Na skladiščnih zemljevidih je razmerje za oba med 0,888 (8 agentov) in 0,913 (32 agentov). Na labirintnih zemljevidih PBS doseže razmerje 0,888 pri 16 agentih, Local Leaders pa 0,960 (le pri uspešnih instancah), PBS je tu boljši, ker rešuje vse instance.

4.7 Pretočnost po tipu zemljevida



Slika 8: Lifelong pretočnost MATS-LP, Local Leaders in Follower po tipu zemljevida (naključni, labirint, skladišče). Na skladiščnih zemljevidih Local Leaders dosega do 10× višjo pretočnost kot MATS-LP.

Slika 8 primerja lifelong pretočnost MATS-LP, Local Leaders in Follower na vseh treh tipih zemljevidov. Na naključnih zemljevidih so si algoritmi po pretočnosti blizu. Na labirintih pa vodi MATS-LP (0,906 pri 32 agentih), Local Leaders pa doseže 0,672 (pri 32 agentih), kar je boljše od Followerja. Na skladiščnih zemljevidih se pa vlogi zamenjata: Local Leaders doseže 1,16 (32 agentov) oziroma 2,14 (96 agentov), MATS-LP pa le 0,11 oziroma 0,28, to je ≈ 8 –10-kratna prednost. Follower na skladiščnih zemljevidih ne uspe zaključiti epizod. Vzrok je v tem, da MCTS v širokih koridorjih postane računsko izjemno zahteven (do 1240 s na epizodo), medtem ko Local Leaders s C++ implementacijo isto epizodo opravi v pod 2 s.

4.8 Povzetek rezultatov

Tabela 1: Povzetek primerjave po metrikah. ++ zelo dobro, + dobro, ~ primerljivo, - slabo, -- zelo slabo / N/A.

Metrika	LaCAM	PBS	LL	MATS-LP	Follower
SoC (one-shot)	~	+	+	-	-
Makespan (one-shot)	+	+	+	-	-
Čas načrtov. (MovingAI)	++	+	-	-	-
Uspešnost (random)	++	++	++	-	-
Uspešnost (maze)	++	++	-	-	-
Pretočnost (0 % ovire)	-	-	~	+	~
Pretočnost (30 % ovire)	-	-	~	+	~
Pretočnost (warehouse)	-	-	++	-	-
Čas epizode (POGEMA)	-	-	++	-	+

Preglednica 1 povzema primerjavo vseh petih algoritmov. PBS se izkaže kot izredno kompetenten centraliziran algoritem: na vseh tipih zemljevidov doseže 100 % uspešnost (vključno z labirinti, kjer Local Leaders zataji pri 33–50 %), z SoC, ki je ≈ 9 –11 % nižji kot pri LaCAM, in časom načrtovanja 0,3–20 ms. S tem zapolni vrzel, ki jo pušča Local Leaders v gostih labirintnih scenarijih, brez žrtve pri hitrosti.

Local Leaders se izkaže kot učinkovit pristop: na lifelong scenarijih dosega 97–101 % pretočnosti MATS-LP na odprtih naključnih okoljih, na skladiščnih okoljih pa ga ≈ 8 –10× prekaša, pri čemer je 100–400× hitrejši od MATS-LP po času epizode. Dvofazna razrešitev konfliktov z vodji bistveno izboljša pretočnost v labirintih

(+23 % pri 32 agentih) in pri 30 % ovirah (0,51 vs. 0,90 MATS-LP pri 32 agentih). Follower, kot naučena politika, leži med njima, a ne uspe zaključiti epizod na skladiščnih zemljevidih.

5 ZAKLJUČEK

V tem delu smo predstavili algoritem *Local Leaders*, ki je naš pri-stop za reševanje problema večagentnega iskanja poti. Ta temelju na dinamičnem oblikovanju hierarhije lokalnih skupin z vodji, ne-odvisnem načrtovanju z algoritmom A^* in reševanju konfliktov z uporabo prioritete, vso načrtovanje pa se izvaja sproti.

Naši eksperimenti na zbirkah MovingAI in POGEMA so pokazali:

- **Kakovost rešitev:** Local Leaders premaga LaCAM v metriki Soc, (do 9 % manj) in dosega primerljiv makespan na naključnih in skladiščnih, kar je za lokalni algoritem prese- netljivo.
- **Pretočnost:** Na odprtih (0 % ovir) naključnih okoljih dosega Local Leaders 97–101 % pretočnosti MATS-LP. Na labirintih doseže 0,672 pri 32 agentih (+23 % v primerjavi z osnovno različico). Na skladiščnih okoljih prekaša MATS-LP za 8–10- kratnik (2,14 vs. 0,28 ciljev/korak pri 96 agentih). Pri 30 % ovirah dosega 0,49–0,60 ciljev/korak, kar je bistveno boljše od osnove (bila 0,14–0,28).
- **Hitrost:** Local Leaders je hitrejši od MATS-LP za faktor 100–400, pri čemer opravi epizodo v 0,26–0,97 s, medtem ko MATS-LP potrebuje 48–416 s za isto epizodo, kar potrjuje skalabilnost pristopa.
- **Slabosti:** Algoritem pri 32 agentih in 30 % ovirah zaostaja za MATS-LP (0,51 vs. 0,90), v labirintnih okoljih pa PBS ponuja zanesljivejšo alternativo z zagotovljeno 100 % uspešnostjo.

Našo hipotezo, da hibridni pristop z lokalnimi vodji doseže boljše razmerje med časom načrtovanja in kakovostjo pridobljenih rešitev kot obstoječi centralizirani ali decentralizirani pristopi, potrjujemo za odprta, gosta in skladiščna okolja. Dvofazna razrešitev konfliktov z vodji in posodobitev parametrov ($t_{esc} = 1$) sta bistveno izboljšali vedenje pri visoki gostoti ovir in v labirintih. Za scenarije z ozkimi hodniki bo potrebno razviti zanesljivejši mehanizem razrešitve za- stojev, npr. kratkovidno kooperativno načrtovanje med sosednjimi vodji.

LITERATURA

- [1] Anton Andreychuk, Konstantin Yakovlev, Alexey Skrynnik, and Aleksandr Panov. 2024. Follower: Learning to Solve Decentralized Lifelong Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press.
- [2] Mehul Damani, Zhiyao Luo, Emerson Wenzel, and Guillaume Sartoretti. 2021. PRIMAL₂: Pathfinding via Reinforcement and Imitation Multi-Agent Learning – Lifelong. *IEEE Robotics and Automation Letters* 6, 2 (2021), 2666–2673. <https://doi.org/10.1109/LRA.2021.3062803>
- [3] Jiaoyang Li, Daniel Harabor, Peter J. Stuckey, Ariel Felner, Hang Ma, and Sven Koenig. 2019. Disjoint Splitting for Multi-Agent Path Finding with Conflict-Based Search. 29 (Jul. 2019), 279–283. <https://doi.org/10.1609/icaps.v29i1.3487>
- [4] Jiaoyang Li, Wheeler Ruml, and Sven Koenig. 2021. EECBS: A Bounded-Suboptimal Search for Multi-Agent Path Finding. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 12353–12362. <https://doi.org/10.1609/aaai.v35i14.17466>
- [5] Hang Ma, Daniel Harabor, Peter J. Stuckey, Jiaoyang Li, and Sven Koenig. 2019. Searching with consistent prioritization for multi-agent path finding. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence (Honolulu, Hawaii, USA) (AAAI’19/IAAI’19/EAAI’19)*. AAAI Press, Article 938, 8 pages. <https://doi.org/10.1609/aaai.v33i01.33017643>
- [6] Keisuke Okumura. 2023. LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. AAAI Press, 11655–11662. <https://doi.org/10.1609/aaai.v37i10.26377>
- [7] Alexey Skrynnik, Anton Andreychuk, Anatolii Borzilov, Alexander Chernyavskiy, Konstantin Yakovlev, and Aleksandr Panov. 2025. POGEMA: A Benchmark Platform for Cooperative Multi-Agent Pathfinding. In *The Thirteenth International Conference on Learning Representations*.
- [8] Alexey Skrynnik, Anton Andreychuk, Konstantin Yakovlev, and Aleksandr Panov. 2023. Decentralized Monte Carlo Tree Search for Partially Observable Multi-agent Pathfinding. *arXiv preprint arXiv:2312.15908* (2023).
- [9] Alexey Skrynnik, Anton Andreychuk, Konstantin Yakovlev, and Aleksandr Panov. 2024. Decentralized Monte Carlo Tree Search for Partially Observable Multi-Agent Pathfinding. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence (AAAI-24)*. AAAI Press.
- [10] Roni Stern, Nathan Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, T. Kumar, and Eli Boyarski. 2021. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. *Proceedings of the International Symposium on Combinatorial Search* 10 (09 2021), 151–158. <https://doi.org/10.1609/socs.v10i1.18510>
- [11] Yutong Wang, Bairan Xiang, Shao Huang, and Guillaume Sartoretti. 2023. SCRIMP: Scalable Communication for Reinforcement- and Imitation-Learning-Based Multi-Agent Pathfinding. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. <https://doi.org/10.1109/IROS55552.2023.10341379>